

Building an MCP Server for Your Custom Web Components

By Sudeep Parchure — April 2026

What is MCP?

MCP stands for **Model Context Protocol**. It is an open standard, introduced by Anthropic in late 2024, that defines how AI assistants communicate with external tools, data sources, and services.

Think of it like a **USB standard for AI**. Before USB, every device needed a different connector. Before MCP, every AI tool needed a custom integration. MCP fixes that by giving every AI assistant one consistent way to call any tool.

When you configure an MCP server in VS Code or Claude Desktop, the AI can call your tools the same way a developer calls an API — with defined inputs, outputs, and descriptions.

Why MCP Exists

AI assistants are smart, but they are blind to your private systems. Without a structured way to connect them, every team ends up building one-off plugins — fragile, undocumented, and tied to a single AI product.

MCP solves three real problems:

- **Discovery** — the AI can ask "what tools are available?" and get a structured answer
- **Consistency** — one server works with any MCP-compatible client (VS Code, Claude, Cursor, etc.)
- **Portability** — move between AI assistants without rewriting your integrations

In short, MCP turns your internal knowledge — component libraries, APIs, databases — into something an AI can reliably use.

Modern frontend teams build **design systems** — shared component libraries with dozens of components, each with their own props, variants, and usage rules.

The pain is real: developers forget component names, mix up prop signatures, and waste time looking things up in Storybook or Figma. Onboarding new team members takes weeks, not days.

What if your AI coding assistant already knew every component in your library — the props, the snippets, the design tokens — without you having to ask?

That is exactly what this project delivers.

MCP Server vs. Custom AI Agent — Which One to Use?

This is the most common question teams ask when they start exploring AI tooling. The honest answer is: **they solve different problems**.

Use an MCP Server when:

- You have **structured, queryable data** — component manifests, API specs, database records
- You want the tool to work with **any AI assistant**, not just one you build yourself
- You want **fast, lightweight** lookups — not reasoning or decision-making
- You want developers to stay in their IDE and get answers inline
- Setup time matters — MCP servers can be running in under an hour

Use a Custom AI Agent when:

- You need the AI to **take multi-step actions** — e.g. create a ticket, then assign it, then notify Slack
- The task requires **reasoning across multiple sources** that are not structured
- You need a **dedicated chat interface** or a standalone product
- You are building something that must work **without a human in the loop**

The practical rule: if you are augmenting a developer's workflow inside an IDE, use MCP. If you are automating a business process end-to-end, use a custom agent.

For a component library, MCP is the right answer — structured data, instant lookups, works in VS Code today.

The Project: CL Component Library with MCP

GitHub: <https://github.com/sudeep31/UIComponentsWithMCP>

This is a proof-of-concept built in April 2026. It demonstrates the full journey — from design tokens to a working MCP server — across seven phases.

What Is in the Project

`@cl/tokens` — Design tokens (colours, spacing, typography) generated via Style Dictionary.

`@cl/react` — Six React components: Button, TextBox, NumberBox, Select, TextArea, List. Each is typed, themed via tokens, and documented.

`@cl/web-components` — The same six components compiled as standard Web Components (Custom Elements). Framework-agnostic — works in Angular, Vue, plain HTML, or any future framework.

Storybook — Visual playground for all components. Developers can explore every prop and variant interactively.

Docusaurus — Full documentation site with component API reference, getting-started guide, and design token catalogue.

MCP Server — A Python server (FastMCP) that exposes four AI tools: `list_components`, `get_component_props`, `get_component_snippet`, and `get_design_tokens`.

Architecture

The MCP server is intentionally simple. It reads static JSON manifests and returns structured data. There is no database, no network call, no complex dependency.

```
VS Code / Claude / Cursor
|
| JSON-RPC over stdio
|
▼
mcp-server/main.py (FastMCP)
├── list_components      → reads manifests/
├── get_component_props  → reads manifests/<name>.json
├── get_component_snippet → generates ready-to-use code
└── get_design_tokens    → reads tokens/
```

Each component manifest is a plain JSON file that describes the component — its props, their types, defaults, and whether they are required. The MCP server reads these files and exposes them as AI-callable tools.

Installation

Prerequisites

- Python 3.11 or newer
- Node.js 20 or newer
- pnpm 9 or newer (`npm install -g pnpm`)

Step 1 — Clone the repository

```
git clone https://github.com/sudeep31/UIComponentsWithMCP.git
cd UIComponentsWithMCP
```

Step 2 — Install Node dependencies

```
pnpm install
```

Step 3 — Build all packages

```
pnpm --filter @cl/tokens build
pnpm --filter @cl/react build
pnpm --filter @cl/web-components build
```

Step 4 — Set up the MCP server

```
cd mcp-server
python -m venv .venv
.venv\Scripts\activate      # Windows
pip install -r requirements.txt
```

Step 5 — Verify the server

```
python test_tools.py
```

All 24 tests should pass. The server is now ready.

How the MCP Server Works — The Code

The entire server is about 80 lines of Python. Here is the full code with a line-by-line explanation.

What is FastMCP?

FastMCP is not FastAPI. They look similar but serve different purposes.

FastAPI is a web framework — it creates HTTP servers with REST endpoints that browsers and apps call over a network.

FastMCP is an MCP framework — it creates stdio servers that AI assistants call locally using the Model Context Protocol. No HTTP, no ports, no browser. VS Code spawns the Python process directly and communicates with it over standard input/output.

The similarity is the decorator pattern. Both use `@app.tool()` or `@app.get()` style decorators. FastMCP borrows that developer-friendly style and applies it to the MCP protocol.

```
from mcp.server.fastmcp import FastMCP
```

This imports FastMCP from the official `mcp` Python package — the same package Anthropic publishes. `requirements.txt` contains just one line:

```
mcp>=1.0.0
```

That single package brings in everything the server needs — the protocol implementation, the stdio transport, and the FastMCP convenience layer.

Installing the Packages

```
cd mcp-server

# Step 1: Create an isolated Python environment (so packages don't
#         conflict with anything else on your machine)
python -m venv .venv

# Step 2: Activate the environment
.venv\Scripts\activate      # Windows
source .venv/bin/activate   # Mac / Linux

# Step 3: Install everything listed in requirements.txt
pip install -r requirements.txt
```

After this, `.venv/Scripts/python` is a standalone Python interpreter that has `mcp` installed. We point VS Code at this exact executable in `mcp.json` — that is why the path includes `.venv/Scripts/python`.

The Path Arguments Explained

```
_SERVER_DIR      = Path(__file__).parent
_WORKSPACE_ROOT  = _SERVER_DIR.parent
_MANIFESTS_DIR   = _SERVER_DIR / "components" / "manifests"
_TOKENS_FILE     = _WORKSPACE_ROOT / "packages" / "tokens" / "dist" / "tokens.json"
```

`Path(__file__)` is Python's way of saying "the file currently running". It does not matter where you launch Python from — `__file__` always refers to `main.py` itself.

`.parent` moves one level up the folder tree. So:

- `_SERVER_DIR` = the `mcp-server/` folder
- `_WORKSPACE_ROOT` = the root of the whole project (one level above `mcp-server/`)
- `_MANIFESTS_DIR` = `mcp-server/components/manifests/` — where the JSON files live
- `_TOKENS_FILE` = `packages/tokens/dist/tokens.json` — the compiled design tokens

This is important because VS Code can spawn the server from any working directory. Using `__file__` makes every path relative to where `main.py` lives, not where the user happens to be. The server always finds its files regardless of how it is launched.

The Server Instance

```
mcp = FastMCP(
    name="cl-components",
    instructions=(
        "You are an assistant for the CL Component Library. "
        "Use list_components to discover available components, ..."
    ),
)
```

`name` is the identifier VS Code shows in the MCP panel. `instructions` is a system-level prompt that is sent to the AI every time it connects — it tells the AI what the server is for and how to use it. This is why Copilot Chat immediately knows to call `list_components` first without being told.

The Four Tools

```
@mcp.tool()
def list_components() -> list[dict]:
    """List all available CL components."""
    for manifest_path in sorted(_MANIFESTS_DIR.glob("*.json")):
        data = json.loads(manifest_path.read_text(encoding="utf-8"))
        result.append({ "name": data["name"], "tag": data["tag"],
                        "description": data["description"] })

    return result
```

`@mcp.tool()` registers this function as an AI-callable tool. The **docstring** becomes the tool's description — the AI reads it to decide when to call this function. The **return type hint** (`list[dict]`) tells the MCP framework how to validate and serialise the response.

```
@mcp.tool()
def get_component_props(component: str) -> list[dict]:
    manifest = _load_manifest(component)
    return manifest["props"]
```

The `component: str` argument becomes a required parameter the AI must provide. FastMCP reads the type hint and generates the JSON Schema for it automatically — no manual schema writing needed.

```
@mcp.tool()
def get_component_snippet(
    component: str,
    flavor: Literal["react", "webcomponent"] = "react",
) -> str:
    snippets = manifest.get("snippets", {})
    return snippets[flavor]
```

`Literal["react", "webcomponent"]` restricts the argument to exactly those two values. FastMCP converts this into an enum in the JSON Schema, so the AI cannot pass an invalid value. The `= "react"` makes it optional with a sensible default.

```
if __name__ == "__main__":
    mcp.run(transport="stdio")
```

`transport="stdio"` tells FastMCP to read JSON-RPC messages from standard input and write responses to standard output. This is the MCP stdio transport — the simplest way to connect an AI assistant to a local server without opening any network ports.

Component Manifest — `manifests/button.json`

```
{
  "name": "cl-button",
  "tag": "cl-button",
  "description": "Primary action button with variant and loading support",
  "props": [
    { "name": "variant", "type": "string", "default": "primary" },
    { "name": "disabled", "type": "boolean", "default": false },
    { "name": "loading", "type": "boolean", "default": false }
  ],
  "snippets": {
    "react": "<Button variant=\"primary\">Label</Button>",
    "webcomponent": "<cl-button variant=\"primary\">Label</cl-button>"
  }
}
```

The manifest is the only place component knowledge lives. The Python server contains zero component-specific code — it just reads these files. Adding a new component to the library means adding one JSON file. No changes to `main.py` are ever needed.

What Other Tools Can Be Added

The four tools in this POC cover the basics. Here are eight real additions that would make the server production-ready:

`search_components(query: str)` — fuzzy search across all component names and descriptions. Useful when a developer types "text input" and needs to find `cl-textbox`.

`validate_snippet(code: str, component: str)` — checks whether a given snippet uses valid props for that component. The AI could call this automatically before returning code to the user.

`get_changelog(component: str)` — reads a `CHANGELOG.md` per component and returns recent changes. Stops developers using deprecated props.

`get_accessibility_notes(component: str)` — returns ARIA roles, keyboard behaviour, and screen-reader notes stored in the manifest. Useful for teams with accessibility requirements.

`get_design_tokens_for_component(component: str)` — returns only the tokens used by a specific component (e.g. the exact colour and spacing tokens that style `cl-button`), rather than the full token catalogue.

`list_deprecated_props(component: str)` — returns props that are still supported but scheduled for removal, so the AI can warn developers proactively.

`get_storybook_url(component: str)` — returns the direct Storybook URL for a component. The AI can include it as a reference link in its answers.

`suggest_component(description: str)` — given a plain-English description like "I need a dropdown with search", returns the most suitable component from the library. This uses simple keyword matching in the manifests — no separate AI model needed.

Each of these follows the same pattern: a decorated Python function, a clear docstring, typed arguments, and a JSON-serialisable return value. The MCP framework handles everything else.

Connecting to VS Code

Once the server is running, register it in your project's `.vscode/mcp.json`:

```
{
  "servers": {
    "cl-components": {
      "type": "stdio",
      "command": "C:/UIComponentsWithMCP/mcp-server/.env/Scripts/python",
      "args": ["C:/UIComponentsWithMCP/mcp-server/main.py"],
      "cwd": "C:/UIComponentsWithMCP"
    }
  }
}
```

Reload VS Code. Open GitHub Copilot Chat. You can now ask:

- "What components are available in the CL library?"
- "Show me the props for cl-select"
- "Give me a snippet for cl-button in Angular"

The AI answers instantly — from your own library, not from the internet.

Using the Web Components in Any Project

The `@cl/web-components` package compiles every component as a standard Custom Element. It requires no framework.

In plain HTML:

```
<script type="module" src="./dist/custom-elements.js"></script>
<link rel="stylesheet" href="./dist/custom-elements.css" />

<cl-button variant="primary">Save</cl-button>
<cl-textbox label="Email" type="email"></cl-textbox>
```


In Angular:

```
npm link @cl/web-components      # or install from npm
```

```
// main.ts
import "@cl/web-components";

// component.ts
schemas: [CUSTOM_ELEMENTS_SCHEMA];
```

```
<!-- template -->
<cl-button [loading]="isSaving()" (click)="save()">Save</cl-button>
```

The same bundle works in React, Vue, Svelte, or any other framework that supports standard HTML elements.

What I Learned

A few honest observations from building this POC:

MCP servers are simpler than expected. The FastMCP library reduces the server to decorated Python functions. The protocol handles discovery, validation, and transport. You focus on the tools, not the plumbing.

JSON manifests are the right abstraction. Storing component metadata as static JSON means the server starts instantly, needs no database, and is easy to version-control alongside the components themselves.

Web Components solve the framework lock-in problem. Building once in React and exporting as Custom Elements means the same components work in every project — including legacy Angular apps and plain HTML prototypes.

The MCP + Web Components combination is genuinely useful. When Copilot Chat can call `get_component_snippet("cl-button", "angular")` and return accurate code from your actual library, it stops hallucinating component names and prop signatures. That alone saves hours per sprint.

References

- MCP Specification — <https://modelcontextprotocol.io>
 - FastMCP (Python) — <https://github.com/jlowin/fastmcp>
 - Web Components (MDN) — https://developer.mozilla.org/en-US/docs/Web/API/Web_components
 - Style Dictionary — <https://amzn.github.io/style-dictionary>
 - Storybook — <https://storybook.js.org>
 - Angular Custom Elements — <https://angular.dev/guide/elements>
 - VS Code MCP Support — <https://code.visualstudio.com/docs/copilot/chat/mcp-servers>
 - **Project on GitHub** — <https://github.com/sudeep31/UIComponentsWithMCP>
-

If this was useful, feel free to connect, share, or drop a comment.